Initiation au Développement : Algorithmique Fondamentale, Variables et Structures de Contrôle

Thème principal : Les fondements de la logique algorithmique, la gestion de la mémoire RAM via les variables et le contrôle de flux d'exécution (Alternatives et Boucles).

Synthèse exécutive : Ce cours pose les bases fondamentales de la programmation moderne et de la pensée algorithmique pour les étudiants en BUT Informatique. Il résout le problème de la traduction d'un processus logique du monde réel en une suite d'instructions formelles et déterministes exécutables par un ordinateur. La séance traite de la manière dont une machine réserve et manipule la mémoire vive (RAM) à travers le concept de variables, comment elle prend des décisions conditionnelles (Si / Sinon) grâce aux évaluations booléennes, et comment elle automatise la répétition d'instructions à l'aide des structures itératives (Pour et TantQue).

Mémorisation et Manipulation des Données : Les Variables en RAM

En informatique, une variable se distingue radicalement d'une variable mathématique statique. Il s'agit d'un conteneur physique dynamique en mémoire RAM. Lors d'une instruction d'affectation (par exemple x ← x + 1), le processeur réalise deux étapes séquentielles : premièrement, il évalue la valeur de l'expression située à droite du symbole d'affectation en lisant le contenu actuel de la RAM ; deuxièmement, il écrase l'ancien contenu de la case mémoire désignée à gauche par le nouveau résultat. Tenter d'utiliser une variable non initialisée dans une expression force le système à lire des octets résiduels non nettoyés (valeur 'poubelle'), ce qui provoque des comportements imprévisibles et constitue un piège classique en partiel.

- Variable : Emplacement étiqueté en mémoire vive (RAM) destiné à conserver une valeur modifiable au cours de l'exécution.
- Quadruplet d'une variable : Défini par son Nom (identificateur), son Type (format d'encodage et taille en octets), sa Valeur actuelle et son Adresse mémoire physique.
- Typage formel : Détermine la nature des données (Entier, Réel, Chaîne de caractères, Booléen) et restreint les opérations mathématiques ou logiques autorisées.
- Affectation (← ou =) : Opération destructive d'écriture qui évalue l'expression située à droite et remplace sans possibilité de retour la valeur stockée à gauche.

Cas concret enseignant : *La métaphore de l'armoire d'archivage : Imaginez la mémoire RAM comme une grande armoire pleine de tiroirs. Déclarer une variable revient à coller une étiquette sur un tiroir (ex: 'solde_compte') et à lui attribuer une taille spécifique selon son type ('Entier'). L'instruction 'solde_compte ← 150' consiste à ouvrir ce tiroir, jeter le morceau de papier qui s'y trouvait auparavant, et y déposer la nouvelle valeur 150.*

Prise de Décision : Les Structures Conditionnelles (Si / Sinon)

Par défaut, l'ordinateur exécute les instructions de manière séquentielle, ligne par ligne. Les structures conditionnelles permettent d'introduire des branchements logiques adaptatifs. Lors de l'évaluation d'un bloc 'Si (Condition) Alors [Bloc A] Sinon [Bloc B]', la machine teste la condition booléenne : si elle s'évalue à VRAI, le bloc A est exécuté et le bloc B est totalement ignoré. Si elle s'évalue à FAUX, le bloc A est sauté et seul le bloc B est exécuté. Dans la combinaison de conditions logiques, l'opérateur ET nécessite que toutes les sous-conditions soient simultanément vraies, tandis que l'opérateur OU requiert qu'au moins une condition soit vraie.

- Structure d'Alternative (Si ... Alors ... Sinon) : Aiguillage logique détournant le flux d'exécution séquentiel selon le résultat d'un test.
- Prédicat / Expression Booléenne : Proposition logique évaluable uniquement à VRAI (True) ou FAUX (False).
- Opérateurs Logiques (ET, OU, NON) : Connecteurs permettant d'assembler des expressions complexes régies par les tables de vérité.

Cas concret enseignant : *Le cas réel du distributeur automatique de billets (DAB) : Avant d'expulser les billets, l'automate évalue une condition composée : SI (solde_client >= montant_demandé) ET (billets_disponibles >= montant_demandé). Si l'expression globale est VRAIE, le paiement est validé ; SINON, l'opération est interrompue, un message d'erreur est affiché et la carte bancaire est restituée.*

Répétition et Automatisation : Les Structures Itératives (Boucles)

La puissance de calcul informatique repose sur la capacité à exécuter des séries d'instructions répétitives à très haute vitesse. On distingue deux grandes familles de structures itératives. La boucle 'Pour' est prévisible et déterministe : elle gère automatiquement une variable compteur (initialisation, test de borne, incrémentation). La boucle 'TantQue' est dynamique : elle évalue une condition avant chaque passage. Si la condition est VRAIE, le corps de la boucle est exécuté, puis la condition est réévaluée. Si la condition initiale est FAUSSE, la boucle ne s'exécute jamais. La règle d'or d'une boucle 'TantQue' est d'assurer que le corps de la boucle modifie la variable de contrôle pour qu'elle devienne FAUSSE à un moment donné.

- Boucle Pour (FOR) : Structure itérative bornée, utilisée lorsque le nombre précis d'itérations est connu avant le début de la boucle.
- Boucle TantQue (WHILE) : Structure itérative non bornée, répétant un traitement tant qu'une condition logique demeure VRAIE.
- Incrémentation & Sortie de boucle : Modification obligatoire d'au moins un paramètre de la condition dans le corps de la boucle pour éviter le blocage.

Cas concret enseignant : *La métaphore des consignes de l'entraîneur d'athlétisme : Si l'entraîneur vous ordonne 'Fais 10 tours de piste', c'est une boucle POUR (le nombre d'itérations 10 est fixé au départ). S'il vous ordonne 'Cours tant qu'il ne pleut pas', c'est une boucle TANT QUE (la fin de l'exercice dépend d'un facteur météo évalué à chaque tour).*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi l'instruction 'x = x + 1' est-elle valide en programmation alors qu'elle est mathématiquement impossible ?
Réponse du professeur : En programmation, le symbole '=' (ou '←') n'exprime pas une égalité algébrique statique, mais une instruction d'affectation dynamique. Il faut lire l'instruction de droite à gauche : l'ordinateur prend la valeur actuelle stockée dans 'x', lui ajoute 1, puis vient ranger ce nouveau résultat dans la case mémoire 'x', écrasant la précédente valeur.

Question étudiant : Que se passe-t-il concrètement si j'oublie d'actualiser la variable de contrôle dans l'instruction 'TantQue' ?
Réponse du professeur : La condition de boucle restant perpétuellement VRAIE, le programme entre dans ce qu'on appelle une 'boucle infinie'. Il va exécuter les instructions sans jamais s'arrêter, saturant le processeur et provoquant le gel (plantage) de l'application.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un algorithme en pseudo-code qui demande à l'utilisateur de saisir un nombre entier positif N, puis calcule et affiche la somme accumulée de tous les entiers compris entre 1 et N inclus (1 + 2 + ... + N).

Résolution pas à pas :
Étape 1 : Déclaration des variables : - N : Entier (limite saisie par l'utilisateur) - i : Entier (compteur de boucle) - somme : Entier (accumulateur) Étape 2 : Saisie utilisateur : Afficher "Entrez un nombre entier positif N : " Lire N Étape 3 : Initialisation de l'accumulateur : somme ← 0 Étape 4 : Boucle itérative de calcul : Pour i allant de 1 à N par pas de 1 Faire somme ← somme + i FinPour Étape 5 : Affichage de la valeur finale : Afficher "La somme des entiers de 1 à ", N, " est : ", somme

Résultat : Pour une saisie N = 5, le déroulement des états mémoire est le suivant : - Initialisation : somme = 0 - Passages dans la boucle : • i = 1 → somme = 0 + 1 = 1 • i = 2 → somme = 1 + 2 = 3 • i = 3 → somme = 3 + 3 = 6 • i = 4 → somme = 6 + 4 = 10 • i = 5 → somme = 10 + 5 = 15 Résultat affiché par le programme : 15.

L'algorithme est correct et déterministe.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'usage entre une boucle POUR et une boucle TANT QUE ?

R : La boucle POUR est utilisée lorsque le nombre d'itérations est connu à l'avance (itérations bornées). La boucle TANT QUE est utilisée lorsque le nombre d'itérations dépend d'une condition dynamique évaluée à l'exécution (itérations indéterminées).

Q : Que vaut la variable A après l'exécution de la séquence : A ← 5 ; B ← 3 ; A ← A + B ; B ← A - B ?

R : À la fin de l'exécution, A vaut 8. Déroulement step-by-step : A prend 5, B prend 3. Puis A ← 5 + 3 = 8. Enfin B ← 8 - 3 = 5. La valeur finale de A est donc 8.

Q : Pourquoi est-il strictement obligatoire d'initialiser une variable d'accumulation (ex: somme ← 0) avant de l'utiliser dans une boucle ?

R : Si la variable n'est pas initialisée, elle contient une valeur 'poubelle' résiduelle présente en mémoire RAM. La première opération de cumul (somme ← somme + i) ajouterait la valeur de 'i' à ce résidu indéterminé, faussant totalement le résultat.